
PLATO · AI-ENABLED QA & SDET TESTING

Risk-Based Regression Agent

*How to set it up and run it —
step by step, from scratch*

For anyone setting this up for the first time

Time needed about 15 minutes

You need a computer and Python. Nothing else to begin with.

Date September 2026

1 • Introduction

What this tool is and why it exists. One page — then we get to work.

The problem

Running every regression test on every release takes hours. Picking a handful by instinct is quicker, but when something ships broken nobody can explain why a particular test was skipped.

What this tool does

It picks the tests worth running, runs only those, and writes down a reason for every choice — including the ones it skipped. So you get a short test run *and* a record you can defend.

It works two ways

	WAY 1 — PAST PROBLEMS	WAY 2 — RECENT CHANGES
It asks	What has broken before?	What changed this release?
It looks at	Old bug tickets, and which code gets edited a lot	New features, bug fixes and changed requirements
It picks	Tests for the parts that keep breaking	Tests for the parts that just changed

The two ways usually pick **different** tests, and that is the point. One protects you against old weak spots, the other against new work.

What it will and won't do

IT WILL	IT WILL NEVER
Choose and run a smaller set of tests	Make up a test that doesn't exist
Explain every choice, including skips	Change your main branch
Fix a test that has gone out of date	Edit your app to hide a failing test
Raise a Jira ticket when the app is truly broken	Merge anything without a person approving

THE IMPORTANT BIT

When a test fails, the tool decides whether the **test** is out of date or the **app** is broken. An out-of-date test gets a suggested fix for someone to approve. A broken app gets a Jira ticket — the tool does not touch your product code to make a red test go green.

How to use this guide

Follow sections 2 to 4 in order, typing each command exactly as shown. After each command there is a green box telling you what you should see, and an amber box telling you what to do if you don't. If both look right, move on.

2 • Getting the code

Two ways. Pick whichever suits you — the result is identical.

Where the code lives

```
https://github.com/shukkoorkassim/plato-risk-regression-poc
```

Option A — download it from GitHub (no tools needed)

1. Open the address above in a browser.
2. Click the green **Code** button, then **Download ZIP**.
3. Unzip it somewhere easy to find, such as your Desktop.

Option B — clone it with git

If you already have git installed, open a terminal and run:

```
git clone https://github.com/shukkoorkassim/plato-risk-regression-poc.git
cd plato-risk-regression-poc
```

YOU SHOULD SEE

A new folder called `plato-risk-regression-poc`, and your terminal now inside it.

IF IT DIDN'T WORK

"git is not recognised" — git isn't installed. Use Option A instead, or install it from `git-scm.com`.

"repository not found" — the code hasn't been published yet. Ask whoever sent you this guide.

Open a terminal in that folder

Everything from here on is typed into a terminal that is **inside the project folder** — the one containing a file called `run_poc.py`.

ON	DO THIS
Windows	Open the folder, click the address bar at the top, type <code>powershell</code> and press Enter.
Mac	Right-click the folder → Services → New Terminal at Folder.
Linux	Right-click inside the folder → Open Terminal Here.

CHECK YOU ARE IN THE RIGHT PLACE

Type `dir` on Windows, or `ls` on Mac and Linux. You should see `run_poc.py` in the list. If you don't, you are in the wrong folder.

3 • Configuring it

Four steps. Type each line, press Enter, and check the green box before moving on.

1. Check you have Python

```
python --version
```

YOU SHOULD SEE

Something like `Python 3.11.4`. Any 3.9 or higher is fine.

IF IT DIDN'T WORK

Nothing happens, or "not recognised" — try `python3 --version` instead. If that also fails, install Python from `python.org`. On Windows, tick

"ADD PYTHON TO PATH"

during install — this is the step people miss.

It says 3.8 or lower — install a newer version from `python.org`.

2. Make a clean space for it

This keeps the tool's files separate from everything else on your computer. Recommended, but you can skip it.

Windows

```
python -m venv .venv
.venv\Scripts\Activate.ps1
```

Mac or Linux

```
python3 -m venv .venv
source .venv/bin/activate
```

YOU SHOULD SEE

Your prompt now begins with `(.venv)`.

IF IT DIDN'T WORK

Windows says *running scripts is disabled* — run this once, then try again:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

3. Install what it needs

```
python -m pip install -r requirements.txt
```

YOU SHOULD SEE

A list of packages installing, ending with `Successfully installed`. Takes under a minute.

IF IT DIDN'T WORK

"No such file" — you are not in the project folder. Go back to the end of section 2.

Network or timeout errors — check your connection, or your company proxy, and try again.

4. Create your settings file

```
Windows
copy .env.example .env

Mac or Linux
cp .env.example .env
```

YOU SHOULD SEE

`1 file(s) copied` on Windows, or no message at all on Mac and Linux — no message means it worked.

That's it. **You can leave the file completely untouched for now** — every setting in it is optional, and the tool runs on its own sample data. Section 5 explains what to put in it when you're ready.

ONE RULE, FOREVER

Once you put a real password or token in `.env`, never rename it, copy it, email it, or paste its contents into a chat or a ticket.

4 • Running it

Three commands, in order. None of these need an account or an internet connection.

1. Set up the sample data

```
python run_poc.py --reset
```

YOU SHOULD SEE

A short confirmation that the demo has been reset. This puts two deliberate bugs back into the sample app so you can watch the tool find them.

2. Check everything is healthy

```
python run_poc.py --selftest
```

YOU SHOULD SEE

A checklist, mostly ticks. If you haven't added any accounts it will say so — that is correct and expected, not a failure.

IF IT DIDN'T WORK

This command names the exact problem rather than leaving you guessing. Read what it says, then check section 7.

3. Watch it work

```
python run_poc.py --dry-run
```

YOU SHOULD SEE

The tool read the data, score each part of the app, choose a small set of tests, run them, and report failures.

THOSE FAILURES ARE CORRECT

— they are the planted bugs being caught.

Showing it to other people

```
python run_poc.py --showcase --pace 1.0
```

A narrated walk-through in ten stages, paced for an audience. Drop `--pace 1.0` to let it run at full speed.

Comparing the two ways

```
python run_poc.py --report
```

Shows both ways side by side — what each ranked, chose, skipped and ran. This is the clearest single view of what the tool does.

Letting it fix things by itself

These two need a Claude API key in your `.env` (section 5). Everything above works without one.

```
python run_poc.py --flow 1    way 1 - past problems
python run_poc.py --reset    important - see below
python run_poc.py --flow 2    way 2 - recent changes
```

ALWAYS RESET IN BETWEEN

The tool genuinely repairs the sample app. Without `--reset` between the two runs, the second one finds nothing left to fix and looks like it failed.

5 • Settings

Only needed when you want live data or self-healing. Skip this until then.

How the file works

Open `.env` in Notepad or any text editor. Each line is `NAME=value`. A line starting with `#` is switched off — delete the `#` to switch it on. For yes/no settings, `1` means yes.

```
# JIRA_LIVE=1      <- off
JIRA_LIVE=1       <- on
```

The ones most people need

SETTING	WHAT IT DOES
ANTHROPIC_API_KEY	Your Claude key, from platform.claude.com. Needed only for <code>--flow 1</code> and <code>--flow 2</code> . Costs a few cents a run.
RISK_THRESHOLD	How picky the tool is. Lower means more tests run and less chance of missing something; higher means fewer tests and more time saved. Starts at <code>5</code> .
JIRA_LIVE	<code>1</code> reads your real Jira instead of the sample file.
CHANGE_LIVE	<code>1</code> reads this release's changes from Jira and Confluence instead of the sample files.
ATLASSIAN_SITE	Your Atlassian address, like <code>yourcompany.atlassian.net</code> .
ATLASSIAN_EMAIL	The email you sign in to Atlassian with.
ATLASSIAN_TOKEN	Your Atlassian API token, from id.atlassian.com under Security. One token covers both Jira and Confluence.
JIRA_PROJECT_KEY	Your Jira project's short code, like <code>KAN</code> .

Useful when demonstrating

SETTING	WHAT IT DOES
REPORT_BUGS	<code>1</code> lets it create a real Jira ticket when it finds a genuine bug. Leave off while practising, or you'll create one every run.
APPROVE_ALL	<code>1</code> skips the four "a human must approve this" pauses so a demo runs straight through.

Checking a live connection

After switching on `JIRA_LIVE` or `CHANGE_LIVE`, confirm it works before relying on it:

```
python scripts/check_jira.py
python scripts/check_changes.py
```

YOU SHOULD SEE

A count of the tickets and requirements it found. If it says `CSV`, live mode is not on — check section 7.

There are about a dozen more settings for unusual cases. They are all listed and explained inside the `.env` file itself.

6 • Using it on your own project

Moving off the sample data. Do these in order.

1. Write down the parts of your app

Ten to thirty names — `checkout`, `login`, `search`, whatever fits your product. Everything else refers back to this list, so being **consistent** matters far more than being detailed.

2. Say which test covers which part

Open `tests/test_map.json`. It pairs each test file with one part name:

```
{
  "tests/test_login.py": "login",
  "tests/test_checkout.py": "checkout"
}
```

Replace it with your own, then run:

```
python run_poc.py --check-data
```

YOU SHOULD SEE

A report naming anything missing or misspelled. Fix what it lists and run it again until it's clean.

3. Give it your bug history

Either fill in `data/jira_export.csv` with your resolved bugs, or switch on `JIRA_LIVE=1` and let it read Jira directly.

4. Give it this release's changes

Either fill in the four spreadsheets in `data/release_delta/`, or switch on `CHANGE_LIVE=1`.

5. Try it on releases you already know about

Run it against two or three past releases where you remember what broke. Compare what it picked with what actually went wrong, and adjust `RISK_THRESHOLD` until it feels right.

DON'T SKIP STEP 5

The settings were tuned on a sample app. They are a sensible starting point, not a tested fit for your product. Check it against releases you already know before letting it decide anything that matters.

Finding gaps in your test coverage

```
python run_poc.py --coverage
```

Lists the risky parts of your app that have **no test at all**, ranked by how risky they are. The tool can't invent a test, but this is often the more useful answer.

7 • If something goes wrong

Beyond the fixes already given beside each step in sections 2 and 3.

WHAT YOU SEE	WHAT TO DO
It asks for an API key	Only <code>--flow 1</code> and <code>--flow 2</code> need one. Use <code>--dry-run</code> or <code>--showcase</code> to show everything else without a key.
It says CSV when you expected Jira	Live mode is off or a detail is missing. It falls back to the sample file on purpose rather than failing. Check <code>JIRA_LIVE=1</code> and all three Atlassian settings, then run <code>python scripts/check_jira.py</code> .
The second run finds nothing to fix	The first run already fixed it. Run <code>python run_poc.py --reset</code> in between.
It stops and waits for approval	That's intended. Add <code>APPROVE_ALL=1</code> to <code>.env</code> for a demo that doesn't pause.
A new Jira ticket every run	<code>REPORT_BUGS</code> is on. Switch it off while practising.
It picks every test, or none	<code>RISK_THRESHOLD</code> doesn't suit your data. Try a few values and watch <code>--report</code> .
One part is never chosen	It probably has no test listed, or its name is spelled differently in two places. Run <code>python run_poc.py --check-data</code> .
Odd boxes or symbols on screen	An older terminal. Add <code>--ascii</code> to the command.

WHEN IN DOUBT, RUN THIS

`python run_poc.py --selftest` — it checks the packages, the data files, the test list and your live connections in one pass, and names exactly what's wrong.

Publishing your changes back to GitHub

If you've been given write access and want to upload your changes, double-click `push_to_github.bat` in the `risk-Agent` folder. The first time, a browser window will ask you to sign in to GitHub. On Mac or Linux, run this instead:

```
git add -A
git commit -m "My changes"
git push
```

Where to find more

This guide covers the common path. For every data file's format, every setting and the complete command list, see the longer *Installation and User Guide* in the `docs` folder.